

# **SkyThermal**

## **Paragliding Intelligence Platform**

### **Volume 19 - Implementation Roadmap & Sprint Backlog v2**

#### **Technical**

Purpose: convert Volumes 01-18 into an execution-ready build plan for the next-generation paragliding meteo application: weather map, launch intelligence, AI decision support, safety assistant, paid tiers, club/school/event portals, and hardware weather station ecosystem.

Status: Technical planning baseline. Use this as the working implementation plan before creating Jira/Linear tickets, repository milestones, and sprint boards.

# 1. Executive Implementation Summary

This volume translates the product, architecture, database, API, weather-engine, AI/ML, mobile, infrastructure, security, QA, business, legal, local-expert, hardware, rescue, portal, design-system, and analytics volumes into a sequential build plan. The purpose is not to repeat previous architecture; it is to define what gets built first, what must wait, what depends on what, and which acceptance gates prevent unsafe or commercially weak releases.

The recommended execution strategy is to build a narrow but technically correct MVP rather than a broad prototype. The MVP must prove four things: weather data can be ingested reliably, launch-site flyability can be computed and explained, pilots can use the map and launch cards in the field, and paid entitlements can protect premium features without damaging trust.

- Phase 0 establishes repositories, environments, CI/CD, database migrations, design system, coding standards, and project governance.
- Phase 1 builds the public weather map, launch database, site forecast, flyability score, and basic mobile/web experience.
- Phase 2 adds alerts, accounts, paid tiers, AI briefing, richer map layers, and forecast validation.
- Phase 3 adds live tracking, airspace guardian, community observations, local expert playbooks, and digital twin data capture.
- Phase 4 adds ML forecast correction, in-flight safety assistant, rescue mode, club/school/event portals, and hardware station ingestion.
- Phase 5 scales to international launch coverage, commercial partnerships, marketplace, API licensing, and advanced competition/rescue tooling.

## 1.1 Primary delivery principle

Every build item must map to a feature ID from Volume 01, a service from Volume 02, a database object from Volume 03, an endpoint from Volume 04, and an acceptance test from Volume 10. Items that cannot be traced should not enter an implementation sprint until the relevant specification gap is closed.

## 1.2 Release gates

| Gate              | Purpose                                                  | Required evidence                                                          | Owner            |
|-------------------|----------------------------------------------------------|----------------------------------------------------------------------------|------------------|
| Architecture Gate | Stop feature drift and unsafe shortcuts                  | Design doc reference, API contract, database migration, event contract     | Engineering Lead |
| Safety Gate       | Prevent false-green recommendations                      | Hard blockers tested, uncertainty displayed, safety copy approved          | Safety Lead / QA |
| Data Gate         | Prevent licensing or provenance errors                   | Attribution visible, provider license registered, source provenance stored | Data Lead        |
| Security Gate     | Protect location, tracking, billing and admin operations | Threat model checklist, authz tests, audit logging, privacy checks         | Security Lead    |
| Commercial Gate   | Ensure paid features are entitlement-controlled          | Stripe/app-store mapping, entitlement tests, downgrade/upgrade tests       | Product Lead     |
| Operational Gate  | Ensure service can run in production                     | SLOs, alerts, dashboards, backup/restore test                              | DevOps Lead      |

## 2. Program Structure and Workstreams

The platform should be delivered by parallel workstreams with explicit ownership. Workstreams must not work in isolation; the weather engine, map UX, database, API, and QA teams share a weekly integration review because most user-visible features cut across all of them.

| Workstream            | Scope                                                               | Core outputs                                                  | Dependencies               |
|-----------------------|---------------------------------------------------------------------|---------------------------------------------------------------|----------------------------|
| Product & Safety      | Feature priorities, safety copy, pilot workflows, launch playbooks  | PRD changes, release scope, safety rules, beta pilot feedback | All workstreams            |
| Weather & GIS         | Forecast ingestion, derived variables, map tiles, airspace, terrain | Forecast cube, site forecast, weather layers, map services    | Database, API, DevOps      |
| Backend Platform      | API gateway, services, auth, entitlements, alerts, tracking         | REST APIs, WebSockets, queues, event contracts                | Database, Security, QA     |
| Mobile/Web Frontend   | Map shell, site cards, timeline, AI briefing, tracking UI           | React/Flutter apps, design system, offline cache              | API, Design System, QA     |
| AI/ML                 | AI briefing, forecast correction, digital twins, model evaluation   | Feature store, models, prompt grounding, evaluation gates     | Weather, Database, QA      |
| Infrastructure/DevOps | Environments, CI/CD, monitoring, backups, secrets                   | K3s/Kubernetes, Helm, observability, runbooks                 | Backend, Security          |
| Security/Privacy      | Identity, RBAC, live tracking privacy, audit, threat model          | Security controls, test cases, incident playbooks             | All workstreams            |
| QA/Validation         | Automated tests, forecast validation, false-green monitoring        | Test suites, release gates, pilot beta evidence               | All workstreams            |
| Business/Growth       | Subscriptions, clubs, schools, analytics, launch marketing          | Pricing, entitlement plan, conversion funnels                 | Product, Frontend, Backend |

### 2.1 Suggested initial team

| Role                      | Minimum allocation        | First 90-day responsibility                                                                                |
|---------------------------|---------------------------|------------------------------------------------------------------------------------------------------------|
| Technical/Product Founder | 1.0 FTE                   | Own product thesis, scope discipline, safety positioning, investor narrative and integration decisions.    |
| Full-stack Lead           | 1.0 FTE                   | Implement API gateway, core services, database migrations, frontend integration and technical standards.   |
| Weather/GIS Engineer      | 0.5-1.0 FTE               | Implement Open-Meteo/MET Norway ingestion, site forecast generation, map layer definitions and validation. |
| Frontend Engineer         | 1.0 FTE                   | Implement web dashboard, reusable components, MapLibre integration and mobile-responsive UX.               |
| Mobile Engineer           | 0.5-1.0 FTE after MVP map | Implement offline-first mobile tracking, notifications and field UX.                                       |
| DevOps/SRE                | 0.25-0.5 FTE              | Set up environments, monitoring, deployment pipelines, backups and cost controls.                          |
| QA/Safety Tester          | 0.5 FTE                   | Build test cases for weather, flyability, false-green prevention, mobile and API tests.                    |

### 2.2 Repository model

Start with a pragmatic monorepo until service boundaries stabilize. Split only when teams, deployment cadence, or security boundaries require it.

| Path                    | Purpose                                                | Technology                                  |
|-------------------------|--------------------------------------------------------|---------------------------------------------|
| apps/web                | Desktop web dashboard, admin portal and public site    | Next.js or Vite React, MapLibre, TypeScript |
| apps/mobile             | Pilot mobile app and in-flight mode                    | Flutter or React Native                     |
| services/api-gateway    | Public API routing, auth enforcement, rate limits      | FastAPI / NestJS / Go                       |
| services/weather-engine | Ingestion, normalization, derived weather variables    | Python, xarray, cfgrid, Celery/Temporal     |
| services/flyability     | Scoring engine, risk blockers, site forecast synthesis | Python/FastAPI                              |
| services/tracking       | Live telemetry ingestion and WebSocket streams         | Go/FastAPI + Redis Streams/NATS             |
| services/ai-briefing    | Grounded AI briefings and prompt context builder       | Python/FastAPI                              |
| packages/contracts      | OpenAPI specs, JSON schemas, event schemas             | YAML/JSON Schema                            |
| packages/ui             | Shared design tokens and React components              | TypeScript, Storybook                       |
| infra                   | Docker Compose, Helm, Terraform/OpenTofu, runbooks     | Kubernetes/K3s, Helm                        |
| db/migrations           | Schema migrations and seeds                            | Alembic/Flyway                              |

### 3. Target MVP Definition

The MVP should not attempt to clone Windy. It should prove the differentiator: site-specific decision intelligence for paragliding. The MVP must include enough Windy/Ventusky-style weather-map familiarity to make the product understandable, but the commercial value must come from flyability, site ranking, launch details, and alerts.

#### 3.1 MVP feature boundary

| MVP Feature     | Included in MVP                                                                                      | Explicitly deferred                                                        |
|-----------------|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Map shell       | Topographic map, launch markers, weather legend, altitude selector, timeline, basic layer toggles    | High-fidelity global animation, offline global maps, all weather providers |
| Weather data    | Open-Meteo multi-model data, MET Norway for Norway, site-level hourly forecast                       | Raw ECMWF/GFS GRIB ingestion unless needed for paid tier                   |
| Layers          | Surface wind, altitude wind, gusts, cloudbase, thermals proxy, rain, lightning if provider available | Advanced wave/rotor nowcasting, satellite cloud-growth prediction          |
| Launch database | Initial curated launch sites for 1-3 target regions                                                  | Global launch import without local validation                              |
| Flyability      | GO/MAYBE/NO-GO, score, hard blockers, confidence and explanation                                     | ML site digital twin correction                                            |
| AI briefing     | Grounded summary using only fetched forecast/site/playbook data                                      | Free-form conversational AI, route generation beyond validated data        |
| Accounts        | Email/OIDC login, pilot profile, favorites, basic preferences                                        | Social graph, full club management                                         |
| Paid tier       | Stripe checkout, entitlement table, Pro feature gating                                               | App-store subscriptions and enterprise billing                             |
| Alerts          | Saved alert rules for favorite sites and daily digest                                                | In-flight emergency alerting and rescue dashboard                          |
| QA              | Golden sites, false-green tests, API contract tests, smoke tests                                     | Full global weather validation lab                                         |

#### 3.2 MVP success criteria

- A pilot can open the app, find a launch, inspect surface wind, altitude wind, gusts, cloudbase, thermal proxy, rain and lightning risk, and see a GO/MAYBE/NO-GO decision.
- A pilot can filter launch sites within a radius and sort by flyability, confidence, distance and best launch window.
- Every GO recommendation has visible confidence, reasons and hard safety blockers; the app never shows GO when any hard blocker is active.
- Forecast data is cached, traceable to provider, and displayed with provider attribution.
- The system stores forecast snapshots and later observations so forecast skill and digital twin correction can be built after MVP.
- Paid-tier gating works across frontend, API and database, with downgrade and expired subscription behavior tested.
- The platform can operate for at least one target region with production monitoring, backups and on-call runbooks.

#### 3.3 MVP excluded until after first beta

| Excluded item                | Reason                              | Re-entry condition                                                                |
|------------------------------|-------------------------------------|-----------------------------------------------------------------------------------|
| Full rescue dashboard        | Sensitive and high-liability        | Only after live tracking and privacy controls are production-proven               |
| Hardware station program     | Requires field logistics            | Only after launch-site playbooks and station ingestion prototype                  |
| Competition platform         | Operationally complex               | Only after tracking, map and event portal basics exist                            |
| ML forecast correction       | Needs historical error data         | Start collecting data now; train after enough paired forecast/actual observations |
| Global launch coverage       | Quality risk                        | Expand after local expert workflow and moderation exist                           |
| Native mobile in-flight mode | Battery/location/privacy complexity | Begin after web/mobile PWA map validates demand                                   |

## 4. Phase Plan

| Phase | Name                             | Window      | Exit deliverable                                                                                                                                 |
|-------|----------------------------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| 0     | Foundation                       | Weeks 1-2   | Repos, CI/CD, local/dev/staging environments, database migrations, design tokens, issue templates, coding standards, architecture review process |
| 1     | MVP Weather Map & Site Forecast  | Weeks 3-8   | Map shell, launch database, Open-Meteo/MET Norway ingestion, site forecast, map layers, first flyability API                                     |
| 2     | Accounts, Alerts & Paid Tier     | Weeks 9-12  | Authentication, favorites, pilot profile, alert rules, Stripe, entitlements, Pro gating, email/push notification base                            |
| 3     | AI Briefing & Validation         | Weeks 13-16 | Grounded AI briefing, forecast confidence, provider attribution, forecast validation dashboard, false-green tests                                |
| 4     | Beta Launch & Local Intelligence | Weeks 17-22 | Pilot beta, local expert playbooks, verified reports, community observations, ranking refinements, mobile PWA hardening                          |
| 5     | Post-MVP Expansion               | Weeks 23-36 | Live tracking, native mobile, airspace guardian, digital twin data products, club/school/event MVP portals                                       |
| 6     | Advanced Platform                | Months 9-18 | ML correction, hardware stations, rescue mode, competition operations, API licensing, marketplace and global expansion                           |

### 4.1 Dependency logic

Do not start advanced features before their data collection primitives exist. Digital twins, ML correction, rescue mode and competition features depend on clean telemetry, observations, forecast snapshots, and privacy/consent infrastructure. The build order is intentionally biased toward data capture and correctness over visual scope.

| Dependency                      | Required before                                                            | Reason                                                                              |
|---------------------------------|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Launch database + site geometry | Flyability, local playbooks, alerts, ranking                               | All decisions are site-specific and require validated launch/landing metadata.      |
| Forecast snapshot storage       | Validation, digital twin, AI briefing provenance, paid confidence features | Historical forecast values must be preserved; provider APIs alone are insufficient. |
| Hard safety blocker engine      | GO/MAYBE/NO-GO and AI briefing                                             | Safety-critical wording depends on blocker state.                                   |
| Entitlement service             | Premium layers, AI briefing, club/school features                          | Client-side gating alone is not secure.                                             |
| Consent and privacy model       | Live tracking, rescue mode, community reports                              | Location data requires explicit and auditable consent.                              |
| Observation ingestion           | ML correction and local reality layer                                      | Forecast correction needs paired forecast-vs-actual data.                           |
| Monitoring and SLOs             | Paid launch and beta expansion                                             | Commercial users need reliable service and incident response.                       |

### 4.2 Stage-gate checklist

| Gate                       | Pass condition                                                                                                      |
|----------------------------|---------------------------------------------------------------------------------------------------------------------|
| Alpha internal             | End-to-end site forecast loads for target region; no manual database edits required for normal user flow.           |
| Private beta               | At least 20-50 pilots can use favorite sites, flyability, timeline and alerts; false-green test suite passes.       |
| Public MVP                 | Payment, entitlements, attribution, terms acceptance, monitoring, backup and support workflow are production-ready. |
| Pro+ readiness             | Model comparison, confidence, AI briefing, and advanced layers are stable and clearly differentiated.               |
| Club/school readiness      | Organization roles, portal permissions, audit logs and billing workflows pass security review.                      |
| In-flight safety readiness | Background location, offline rules, audio alerts, privacy consent and emergency sharing pass mobile QA.             |

## 5. Sprint Plan - First 24 Weeks

The sprint plan assumes 2-week sprints. If the team is one person plus AI assistance, keep the same order but stretch each sprint to 3-4 weeks and reduce parallel work. Each sprint should produce a demoable increment and a migration/test artifact.

| Sprint    | Window      | Theme                       | Planned scope                                                                                              | Demo / exit criterion                                                                             |
|-----------|-------------|-----------------------------|------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Sprint 0  | Weeks 1-2   | Foundation                  | Monorepo, local dev, Docker Compose, base schemas, design tokens, API skeleton, CI checks, issue templates | Dev environment runs from clean clone; health endpoints pass; first migration applies/rolls back. |
| Sprint 1  | Weeks 3-4   | Launch database + map shell | Launch table, site seed data, map UI, site markers, search, favorite sites, basic layer menu               | User can open map, search site, view launch card and favorite it.                                 |
| Sprint 2  | Weeks 5-6   | Weather ingestion MVP       | Open-Meteo/MET Norway adapters, forecast snapshot storage, site forecast API, attribution                  | Hourly forecast appears for target sites with provider provenance.                                |
| Sprint 3  | Weeks 7-8   | Flyability v1               | Scoring service, hard blockers, confidence field, explanation list, timeline rows                          | GO/MAYBE/NO-GO result is reproducible from stored forecast values.                                |
| Sprint 4  | Weeks 9-10  | Accounts and preferences    | OIDC/email login, user profile, pilot skill fields, favorite sites, saved units                            | Authenticated user has persistent profile and favorites.                                          |
| Sprint 5  | Weeks 11-12 | Alerts and paid tiers       | Alert rule CRUD, daily digest job, Stripe checkout, entitlement checks, Pro layer gating                   | Paid feature cannot be accessed without valid entitlement; alerts deliver in test environment.    |
| Sprint 6  | Weeks 13-14 | AI briefing v1              | Context builder, grounded AI prompt, evidence bundle, refusal rules, audit storage                         | AI briefing cites forecast fields and refuses when data is stale or missing.                      |
| Sprint 7  | Weeks 15-16 | Forecast validation         | Observation ingestion seed, validation dashboard, model disagreement, forecast confidence                  | Internal dashboard shows forecast error by site/layer/provider.                                   |
| Sprint 8  | Weeks 17-18 | Community reports           | Pilot report API, verified report flags, moderation queue, reality layer on launch card                    | Beta pilot can submit conditions; moderators approve/reject reports.                              |
| Sprint 9  | Weeks 19-20 | Local playbooks             | Site rules DSL, hazard notes, rotor/danger zones, expert approval lifecycle                                | A launch can display site-specific safe wind and danger rules.                                    |
| Sprint 10 | Weeks 21-22 | Beta hardening              | Bug fixing, performance budget, mobile responsive pass, onboarding, terms/attribution polish               | Private beta release candidate passes smoke, QA and safety gates.                                 |
| Sprint 11 | Weeks 23-24 | MVP public release          | Production deployment, billing run-through, support workflows, public landing page, analytics dashboards   | Public MVP launch checklist complete; rollback and backup restore tested.                         |

### 5.1 Sprint ceremony model

- Sprint planning: confirm objective, non-goals, acceptance criteria and data migrations before coding starts.
- Mid-sprint integration review: run the current end-to-end flow in staging and update dependency risks.
- Safety review: every sprint that touches scoring, alerts, AI, tracking or rescue must include a false-green review.
- Demo: demonstrate user value, not only internal APIs. Every sprint must have a visible or measurable output.
- Retrospective: record blocked dependencies, technical debt, QA failures and next-sprint corrective actions.

## 6. Epic Backlog

The backlog below is designed to be imported into an issue tracker. IDs are stable and can be referenced from tickets, commits, tests and release notes. Priority uses P0 for release-blocking, P1 for MVP/V1, P2 for important later, and P3 for optional/expansion.

| Epic ID | Epic                         | Priority | Description                                                                                             | Primary spec    |
|---------|------------------------------|----------|---------------------------------------------------------------------------------------------------------|-----------------|
| E-001   | Platform foundation          | P0       | Create monorepo, environments, service skeletons, coding standards, CI/CD, architecture review workflow | Volume 08       |
| E-002   | Core domain schema           | P0       | Implement users, pilot profiles, launches, forecast snapshots, site forecasts, scores and entitlements  | Volume 03       |
| E-003   | Map shell and layer registry | P0       | Implement Windy/Ventusky-style map shell, layer selector, altitude selector and timeline                | Volume 07/17    |
| E-004   | Weather ingestion MVP        | P0       | Implement Open-Meteo and MET Norway adapters, provider registry, attribution and cache strategy         | Volume 05/12    |
| E-005   | Site forecast API            | P0       | Normalize forecast variables and expose hourly site forecast with provenance and freshness              | Volume 04/05    |
| E-006   | Flyability and risk v1       | P0       | Compute GO/MAYBE/NO-GO, score, confidence, blockers and explanation reasons                             | Volume 01/05/10 |
| E-007   | Launch discovery             | P1       | Rank nearby sites by flyability, drive-worthiness, confidence, distance and best window                 | Volume 01/07    |
| E-008   | Pilot profiles               | P1       | Store skill level, wing class, units, safety preferences and notification preferences                   | Volume 01/03    |
| E-009   | Alerts and notifications     | P1       | Allow favorite-site alerts, daily digest, trigger rules and delivery audit                              | Volume 04/08/10 |
| E-010   | Entitlements and billing     | P1       | Implement Stripe checkout, customer portal, entitlement service and tier gating                         | Volume 11/12    |
| E-011   | AI flight briefing v1        | P1       | Implement grounded context builder, AI response policy, evidence bundle and safety refusal rules        | Volume 06/12    |
| E-012   | Forecast validation          | P1       | Measure forecast error, model disagreement, confidence, stale data and provider performance             | Volume 05/18    |
| E-013   | Community reality layer      | P1       | Pilot reports, verified observations, moderation and local condition summaries                          | Volume 13/18    |
| E-014   | Launch-site playbooks        | P1       | Site rules DSL, local expert approval, hazard zones and safe-wind rules                                 | Volume 13       |
| E-015   | Mobile PWA hardening         | P1       | Responsive map, offline cache basics, installable PWA, mobile-friendly launch cards                     | Volume 07/17    |
| E-016   | Observability and operations | P0       | Metrics, logs, traces, dashboards, SLOs, backup/restore and incident runbooks                           | Volume 08/10    |
| E-017   | Security baseline            | P0       | OIDC/JWT, RBAC, BOLA tests, audit logs, privacy consent and rate limits                                 | Volume 09       |
| E-018   | Analytics pipeline           | P1       | Event taxonomy, product metrics, forecast accuracy metrics, funnel and revenue dashboards               | Volume 18       |
| E-019   | Native mobile app            | P2       | Build native field experience, background location, push, offline maps and tracking                     | Volume 07/15    |
| E-020   | Live tracking                | P2       | Telemetry ingestion, WebSockets, privacy rules, track replay and sharing links                          | Volume 15       |
| E-021   | Airspace guardian            | P2       | OpenAIP ingestion, airspace map overlays and altitude/location warnings                                 | Volume 04/15    |
| E-022   | ML forecast correction       | P2       | Feature store, paired observations, model training, model registry and correction API                   | Volume 06       |
| E-023   | Hardware station ingestion   | P2       | MQTT/webhook ingestion, station health, QC rules and live wind integration                              | Volume 14       |
| E-024   | Club portal                  | P2       | Organization roles, member management, shared sites, safety officer workflows                           | Volume 16       |
| E-025   | School portal                | P2       | Instructor dashboard, student sessions, skill-specific safety                                           | Volume 16       |

|       |                               |    |                                                                                    |              |
|-------|-------------------------------|----|------------------------------------------------------------------------------------|--------------|
|       |                               |    | recommendations                                                                    |              |
| E-026 | Event portal                  | P2 | Task planning, weather briefing, launch management, competition tracking           | Volume 16    |
| E-027 | Rescue mode                   | P3 | SOS sharing, drift/search-area estimation, rescue dashboard and privacy controls   | Volume 15    |
| E-028 | Marketplace and API licensing | P3 | Commercial API, schools/guides marketplace, partner integrations and billing tiers | Volume 11/16 |



## 7. MVP User Story Backlog

These stories define the first implementation tranche. They are intentionally concrete and testable. Story IDs should be used in commits and test names where practical.

| Story ID | User story                                                                                                  | Priority | Acceptance criterion                                                                   |
|----------|-------------------------------------------------------------------------------------------------------------|----------|----------------------------------------------------------------------------------------|
| US-001   | As a pilot, I can search for a launch site so I can inspect conditions before driving.                      | P0       | Search returns name, country, distance and current flyability.                         |
| US-002   | As a pilot, I can view a launch on the map so I can understand terrain and location.                        | P0       | Map centers on selected launch and displays marker, landing reference and coordinates. |
| US-003   | As a pilot, I can toggle surface wind so I can judge launch conditions.                                     | P0       | Layer state changes without page reload and legend updates.                            |
| US-004   | As a pilot, I can select wind at 500/1000/1500/2000 m so I can evaluate drift and XC potential.             | P0       | Altitude selector changes forecast query and label.                                    |
| US-005   | As a pilot, I can view gusts so I can avoid collapse-prone conditions.                                      | P0       | Gust value and gust-spread warning are visible on launch card.                         |
| US-006   | As a pilot, I can see thermals and cloudbase so I can plan soaring window.                                  | P0       | Thermal proxy and cloudbase rows are shown hourly.                                     |
| US-007   | As a pilot, I can see CAPE, rain and lightning risk so I can avoid overdevelopment.                         | P1       | Thunderstorm/rain blockers appear before GO can be displayed.                          |
| US-008   | As a pilot, I can see visibility and temperature/dewpoint so I can evaluate cloud formation and navigation. | P1       | Values are shown with units and stale-data state.                                      |
| US-009   | As a pilot, I can see wave/rotor and foehn flags so I can identify mountain danger.                         | P1       | Flags show MAYBE/NO-GO reasons and source uncertainty.                                 |
| US-010   | As a pilot, I receive a GO/MAYBE/NO-GO status so I do not need to interpret raw layers alone.               | P0       | Status has score, blockers, confidence and explanation.                                |
| US-011   | As a pilot, I can see best launch window so I know when to arrive.                                          | P0       | Window is calculated from hourly score and displayed on card/timeline.                 |
| US-012   | As a pilot, I can compare nearby sites so I can decide where to fly.                                        | P1       | Ranking supports flyability, distance, confidence and best window.                     |
| US-013   | As a pilot, I can save favorite sites so I can quickly check them.                                          | P1       | Favorites persist across sessions and devices.                                         |
| US-014   | As a pilot, I can set an alert for a site becoming flyable.                                                 | P1       | Alert rule can be created, triggered and audited.                                      |
| US-015   | As a pilot, I can read an AI briefing grounded in forecast and site rules.                                  | P1       | Briefing refuses stale/missing data and stores evidence bundle.                        |
| US-016   | As a product operator, I can register a weather data source and attribution.                                | P0       | Provider metadata appears in forecast response and UI attribution.                     |
| US-017   | As a QA tester, I can replay golden weather cases to verify false-green prevention.                         | P0       | Test suite blocks release when hard blockers are ignored.                              |
| US-018   | As a paid user, I can access Pro layers only when my entitlement is active.                                 | P1       | API and UI both enforce entitlement.                                                   |
| US-019   | As a free user, I can see locked premium features with clear upgrade explanation.                           | P1       | Locked state does not expose premium API data.                                         |
| US-020   | As an admin, I can inspect ingestion status and stale providers.                                            | P0       | Ops dashboard shows last run, rows imported, errors and staleness.                     |
| US-021   | As a local expert, I can propose a launch playbook rule.                                                    | P1       | Rule enters approval state and does not affect recommendations until approved.         |
| US-022   | As a moderator, I can approve or reject pilot reports.                                                      | P1       | Moderation state changes report visibility and creates audit event.                    |
| US-023   | As a pilot, I can submit actual launch conditions.                                                          | P1       | Report stores wind, gust, direction, cloudbase, thermals, note and confidence.         |
| US-024   | As an operator, I can view forecast error by provider and site.                                             | P1       | Dashboard shows MAE/bias by variable and forecast horizon.                             |
| US-025   | As a user, I can accept updated terms and privacy notices.                                                  | P0       | Terms version acceptance is recorded before paid/tracking features.                    |

### 7.1 Story quality rule

Each story must have a human-visible acceptance criterion, a database/API trace, and at least one automated test. Stories that only say “implement service” or “improve UI” must be split before entering a sprint.

## 8. Technical Task Backlog by Component

### Backend/API

| Task ID | Task                                            | Target | Done requirement                                         |
|---------|-------------------------------------------------|--------|----------------------------------------------------------|
| BAC-001 | Define OpenAPI skeleton and codegen setup       | MVP    | Automated test or manual acceptance attached before Done |
| BAC-002 | Implement auth middleware and entitlement guard | MVP    | Automated test or manual acceptance attached before Done |
| BAC-003 | Create launch CRUD/read APIs                    | MVP    | Automated test or manual acceptance attached before Done |
| BAC-004 | Implement site forecast endpoint                | MVP    | Automated test or manual acceptance attached before Done |
| BAC-005 | Implement flyability endpoint                   | MVP    | Automated test or manual acceptance attached before Done |
| BAC-006 | Implement alert rule CRUD                       | MVP    | Automated test or manual acceptance attached before Done |
| BAC-007 | Implement AI briefing endpoint                  | V1     | Automated test or manual acceptance attached before Done |
| BAC-008 | Implement admin ingestion status endpoint       | V1     | Automated test or manual acceptance attached before Done |
| BAC-009 | Implement rate limiting and audit logging       | V1     | Automated test or manual acceptance attached before Done |

### Weather/GIS

| Task ID | Task                                      | Target | Done requirement                                         |
|---------|-------------------------------------------|--------|----------------------------------------------------------|
| WEA-001 | Create provider registry                  | MVP    | Automated test or manual acceptance attached before Done |
| WEA-002 | Implement Open-Meteo adapter              | MVP    | Automated test or manual acceptance attached before Done |
| WEA-003 | Implement MET Norway adapter              | MVP    | Automated test or manual acceptance attached before Done |
| WEA-004 | Normalize canonical units                 | MVP    | Automated test or manual acceptance attached before Done |
| WEA-005 | Persist model runs and forecast snapshots | MVP    | Automated test or manual acceptance attached before Done |
| WEA-006 | Generate site forecast tables             | MVP    | Automated test or manual acceptance attached before Done |
| WEA-007 | Implement altitude interpolation          | V1     | Automated test or manual acceptance attached before Done |
| WEA-008 | Implement weather freshness checks        | V1     | Automated test or manual acceptance attached before Done |
| WEA-009 | Produce map layer tile/JSON endpoints     | V1     | Automated test or manual acceptance attached before Done |

### Frontend/Web

| Task ID | Task                               | Target | Done requirement                                         |
|---------|------------------------------------|--------|----------------------------------------------------------|
| FRO-001 | Create design tokens               | MVP    | Automated test or manual acceptance attached before Done |
| FRO-002 | Implement map shell                | MVP    | Automated test or manual acceptance attached before Done |
| FRO-003 | Implement layer panel              | MVP    | Automated test or manual acceptance attached before Done |
| FRO-004 | Implement altitude selector        | MVP    | Automated test or manual acceptance attached before Done |
| FRO-005 | Implement right launch card        | MVP    | Automated test or manual acceptance attached before Done |
| FRO-006 | Implement bottom timeline          | MVP    | Automated test or manual acceptance attached before Done |
| FRO-007 | Implement favorites                | V1     | Automated test or manual acceptance attached before Done |
| FRO-008 | Implement locked premium states    | V1     | Automated test or manual acceptance attached before Done |
| FRO-009 | Implement AI briefing card         | V1     | Automated test or manual acceptance attached before Done |
| FRO-010 | Implement responsive mobile layout | V1     | Automated test or manual acceptance attached before Done |

### Database

| Task ID | Task                       | Target | Done requirement                                         |
|---------|----------------------------|--------|----------------------------------------------------------|
| DAT-001 | Create user/profile tables | MVP    | Automated test or manual acceptance attached before Done |
| DAT-002 | Create launch/site tables  | MVP    | Automated test or manual acceptance                      |

|         |                                       |     |                                                          |
|---------|---------------------------------------|-----|----------------------------------------------------------|
|         |                                       |     | attached before Done                                     |
| DAT-003 | Create forecast model run tables      | MVP | Automated test or manual acceptance attached before Done |
| DAT-004 | Create site forecast hypertables      | MVP | Automated test or manual acceptance attached before Done |
| DAT-005 | Create score/result tables            | MVP | Automated test or manual acceptance attached before Done |
| DAT-006 | Create entitlement/billing tables     | MVP | Automated test or manual acceptance attached before Done |
| DAT-007 | Create alert tables                   | V1  | Automated test or manual acceptance attached before Done |
| DAT-008 | Create observation/report tables      | V1  | Automated test or manual acceptance attached before Done |
| DAT-009 | Create indexes and retention policies | V1  | Automated test or manual acceptance attached before Done |

## AI/ML

| Task ID | Task                                 | Target | Done requirement                                         |
|---------|--------------------------------------|--------|----------------------------------------------------------|
| AI/-001 | Create AI context builder            | MVP    | Automated test or manual acceptance attached before Done |
| AI/-002 | Define evidence bundle schema        | MVP    | Automated test or manual acceptance attached before Done |
| AI/-003 | Write safety refusal prompt          | MVP    | Automated test or manual acceptance attached before Done |
| AI/-004 | Implement stale data refusal         | MVP    | Automated test or manual acceptance attached before Done |
| AI/-005 | Log AI outputs and inputs            | MVP    | Automated test or manual acceptance attached before Done |
| AI/-006 | Create forecast validation data mart | MVP    | Automated test or manual acceptance attached before Done |
| AI/-007 | Create model registry placeholder    | V1     | Automated test or manual acceptance attached before Done |
| AI/-008 | Define future feature store entities | V1     | Automated test or manual acceptance attached before Done |

## DevOps

| Task ID | Task                                | Target | Done requirement                                         |
|---------|-------------------------------------|--------|----------------------------------------------------------|
| DEV-001 | Create Docker Compose               | MVP    | Automated test or manual acceptance attached before Done |
| DEV-002 | Create staging deployment           | MVP    | Automated test or manual acceptance attached before Done |
| DEV-003 | Set up Postgres/PostGIS/TimescaleDB | MVP    | Automated test or manual acceptance attached before Done |
| DEV-004 | Set up Redis                        | MVP    | Automated test or manual acceptance attached before Done |
| DEV-005 | Set up object storage               | MVP    | Automated test or manual acceptance attached before Done |
| DEV-006 | Set up CI pipeline                  | MVP    | Automated test or manual acceptance attached before Done |
| DEV-007 | Set up Prometheus/Grafana/Loki      | V1     | Automated test or manual acceptance attached before Done |
| DEV-008 | Set backup job                      | V1     | Automated test or manual acceptance attached before Done |
| DEV-009 | Create runbooks                     | V1     | Automated test or manual acceptance attached before Done |

## QA/Security

| Task ID | Task                           | Target | Done requirement                                         |
|---------|--------------------------------|--------|----------------------------------------------------------|
| QA/-001 | Write API contract tests       | MVP    | Automated test or manual acceptance attached before Done |
| QA/-002 | Write scoring unit tests       | MVP    | Automated test or manual acceptance attached before Done |
| QA/-003 | Write false-green tests        | MVP    | Automated test or manual acceptance attached before Done |
| QA/-004 | Write data staleness tests     | MVP    | Automated test or manual acceptance attached before Done |
| QA/-005 | Write BOLA tests               | MVP    | Automated test or manual acceptance attached before Done |
| QA/-006 | Write entitlement bypass tests | MVP    | Automated test or manual acceptance attached before Done |
| QA/-007 | Write mobile responsive tests  | V1     | Automated test or manual acceptance attached before Done |
| QA/-008 | Write backup restore test      | V1     | Automated test or manual acceptance attached before Done |

|         |                          |    |                                                          |
|---------|--------------------------|----|----------------------------------------------------------|
| QA/-009 | Create release checklist | V1 | Automated test or manual acceptance attached before Done |
|---------|--------------------------|----|----------------------------------------------------------|

## 9. Definition of Done

A ticket is not done when code compiles. It is done when it is safe, observable, tested, documented and traceable.

| Area          | Definition of Done requirement                                                                       |
|---------------|------------------------------------------------------------------------------------------------------|
| Code          | Merged through review, formatted, linted, type-checked and covered by relevant tests.                |
| API           | OpenAPI schema updated; request/response examples added; error cases documented.                     |
| Database      | Migration is reversible where practical; indexes reviewed; seed/test data updated.                   |
| Weather       | Provider provenance stored; units normalized; stale-data behavior tested.                            |
| Safety        | Hard blockers tested; recommendation wording reviewed; false-green test added where relevant.        |
| Security      | Authz enforced server-side; audit event emitted for sensitive actions; no secrets in logs.           |
| Frontend      | Responsive state checked; loading/empty/error/locked states implemented; accessibility labels added. |
| Operations    | Metrics/logs added; dashboard/alert updated for production critical paths.                           |
| Documentation | Relevant volume/backlog ID referenced in ticket and release notes.                                   |

### 9.1 Release checklist

- All P0/P1 tickets assigned to the release are Done.
- OpenAPI contract tests pass in CI.
- Database migrations apply from empty database and production-like snapshot.
- False-green safety test suite passes.
- Weather ingestion has a clean run in staging for the target region.
- Provider attribution appears in UI and API responses.
- Entitlement tests pass for free, Pro, expired and downgraded states.
- Monitoring dashboards show API latency, ingestion freshness, worker failures and forecast staleness.
- Backup/restore drill completed within target RTO/RPO.
- Terms/privacy version is current and acceptance flow is verified.

## 10. Risk Register

| Risk ID | Risk                                         | Severity | Mitigation                                                                           | Owner          |
|---------|----------------------------------------------|----------|--------------------------------------------------------------------------------------|----------------|
| R-001   | False GO recommendation                      | High     | Hard blockers, false-green tests, confidence display, safety copy, pilot beta review | Product/Safety |
| R-002   | Weather provider licensing violation         | High     | Data-source registry, attribution service, legal review, provenance logs             | Data/Legal     |
| R-003   | Forecast ingestion instability               | Medium   | Retry/backoff, stale-data fallback, provider health dashboard, alerting              | Weather/DevOps |
| R-004   | Paid tier unclear to users                   | Medium   | Transparent locked states, honest value proposition, Pro trial, conversion analytics | Product/Growth |
| R-005   | Location privacy issue                       | High     | Consent events, private default tracking, retention controls, audit logs             | Security       |
| R-006   | Map performance too slow on mobile           | Medium   | Layer budget, vector tile caching, progressive loading, FPS metrics                  | Frontend       |
| R-007   | AI hallucination                             | High     | Grounded context only, refusal policy, evidence logging, evaluation tests            | AI/QA          |
| R-008   | Launch database quality poor                 | High     | Local expert approval, playbook lifecycle, moderation and trust score                | Local Ops      |
| R-009   | Single-founder overload                      | Medium   | Narrow MVP, external contractors for GIS/mobile, strict backlog pruning              | Founder        |
| R-010   | Hardware program distracts from software MVP | Medium   | Keep hardware as post-MVP ingestion prototype only                                   | Product        |

## 11. Milestone Acceptance Gates

| Milestone                | Target evidence                                                              | Go/No-Go decision                        |
|--------------------------|------------------------------------------------------------------------------|------------------------------------------|
| M0 - Foundation complete | Clean clone runs local stack; CI passes; first migration; staging reachable  | Proceed to weather/site implementation   |
| M1 - Weather map alpha   | Map, launch card, provider forecast and layer toggles work for target region | Proceed to scoring and pilot workflows   |
| M2 - Flyability alpha    | GO/MAYBE/NO-GO stable; hard blockers tested; explanation shown               | Proceed to accounts, alerts and payments |
| M3 - Paid beta           | Auth, entitlements, alerts and billing work in staging                       | Invite private beta pilots               |
| M4 - AI beta             | AI briefing grounded, refusals tested, evidence logged                       | Enable AI for limited Pro beta           |
| M5 - Public MVP          | Monitoring, legal, privacy, support, backups and pricing pages ready         | Public launch                            |
| M6 - V1 expansion        | Reports/playbooks/validation stable and enough observations collected        | Begin ML correction and live tracking    |

## 12. Build Order Map

The following order should be treated as the implementation spine. Teams can parallelize inside each band, but should not skip bands because later features depend on earlier data, security and operational primitives.

| Order | Build item                                                 | Why it comes here                                                    |
|-------|------------------------------------------------------------|----------------------------------------------------------------------|
| 1     | Foundation, schemas, CI/CD, staging                        | Everything else depends on reliable build/deploy/migrate loops.      |
| 2     | Launch database and map shell                              | Gives a visible product surface and domain object for all decisions. |
| 3     | Weather ingestion and site forecast                        | Creates the data used by every safety/commercial feature.            |
| 4     | Flyability and risk v1                                     | Establishes differentiation over generic weather apps.               |
| 5     | Accounts, favorites and pilot profile                      | Enables personalization and retention.                               |
| 6     | Alerts and paid tiers                                      | Enables recurring revenue and repeat use.                            |
| 7     | AI briefing and forecast confidence                        | Improves comprehension while remaining grounded in data.             |
| 8     | Validation, reports and playbooks                          | Creates reality feedback loop and local trust.                       |
| 9     | Mobile/offline/tracking                                    | Extends from planning to field use.                                  |
| 10    | ML correction, hardware, clubs, schools, rescue and events | Requires enough usage, data and operational maturity.                |

## 13. MVP Sprint Board Seed

| Status | Epic  | Ticket                                           | Priority | Team            |
|--------|-------|--------------------------------------------------|----------|-----------------|
| To Do  | E-001 | Create monorepo and local stack                  | P0       | Platform        |
| To Do  | E-002 | Implement base database migrations               | P0       | Database        |
| To Do  | E-003 | Implement map shell layout from frontend starter | P0       | Frontend        |
| To Do  | E-004 | Implement Open-Meteo adapter                     | P0       | Weather         |
| To Do  | E-005 | Create site forecast endpoint                    | P0       | Backend         |
| To Do  | E-006 | Implement flyability scoring v1                  | P0       | Backend/Weather |
| To Do  | E-017 | Set up OIDC/JWT auth baseline                    | P0       | Security        |
| To Do  | E-016 | Set up ingestion freshness dashboard             | P0       | DevOps          |
| To Do  | E-010 | Implement entitlement table and Pro layer guard  | P1       | Billing         |
| To Do  | E-011 | Implement grounded AI context builder            | P1       | AI              |
| To Do  | E-012 | Create forecast validation dashboard             | P1       | Analytics       |
| To Do  | E-014 | Create site playbook approval workflow           | P1       | Local Ops       |

## 14. Handoff to Volume 20

Volume 20 should become the Master Index and Combined Architecture Package. It should not introduce major new product features. Its purpose is to connect all previous volumes into one navigable package with cross-reference matrices, document map, build dependency graph, and final developer onboarding sequence.

- Create a feature-to-volume matrix across Volumes 01-19.
- Create feature-to-service, feature-to-database, feature-to-API and feature-to-test matrices.
- List all generated artifacts and their current status.
- Define the final MVP build order and production-readiness checklist.
- Add developer onboarding instructions: local setup, environments, credentials, data seeds and first tasks.

## Appendix A - Issue Template

| Field                | Template value                                                           |
|----------------------|--------------------------------------------------------------------------|
| Title                | [Feature ID] concise action, e.g., [E-006] Implement gust spread blocker |
| Problem              | What user, safety, business or operational problem this solves           |
| Spec references      | Volume number, section, API endpoint, database table and test case       |
| Acceptance criteria  | Bullet list with measurable pass/fail criteria                           |
| Non-goals            | What this ticket must not expand into                                    |
| Data/security impact | Tables changed, events emitted, privacy/security concerns                |
| Test plan            | Unit/integration/e2e/QA checks to run                                    |
| Rollback plan        | How to disable or revert safely                                          |

## Appendix B - First Public MVP Checklist

- Production domain, SSL, CDN and API gateway configured.
- Provider attribution visible on forecast screens.
- Terms, privacy and safety disclaimer accepted by users.
- Hard safety blockers verified against golden test cases.
- Paid tier and downgrade states tested.
- Forecast ingestion stale-data warning tested.

- AI briefing disabled or refusing when evidence bundle is missing.
- Backups and restore drill completed.
- Monitoring alerts delivered to operator channel.
- Support email/help page live.
- Beta feedback form and incident-report form live.
- Rollback plan documented and tested.